Table of Contents

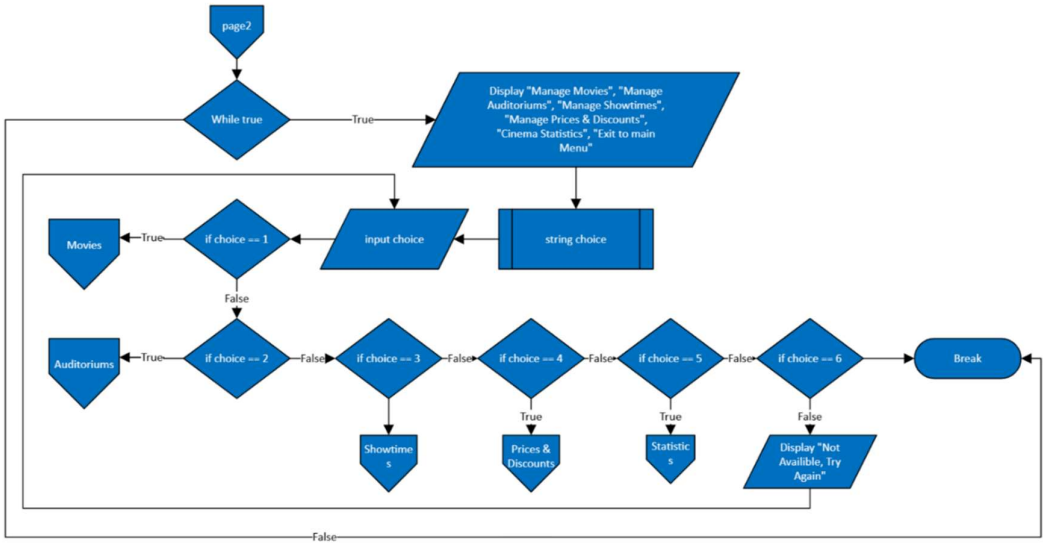
Assumptions About The System

- Movie ids are assigned incrementally m001, m002,...; showtimes will be s001...; auditoriums will be a001...; since user inputs come from upper casing on read.
- Movie titles and auditorium names are uppercase and 1-100 characters in length. If failed, action is cancelled. Durations & capacity
- Movie duration is expected to be integer 30-300 (up to 3 attempts).
- Auditoriums capacity is expected to be 30-1000.
- Hours must end with am/pm; if minutes are omitted, :00 is appended.
- Valid hours input are 12:00pm-1:59am (e.g., 1am is valid); up to 5 attempts.
- When creating a showtime, that must correspond to an existing movie id and an existing auditorium id, when updating, it validates if anything is created newly; when delete/searching, it's strictly by id.
- You cannot delete an auditorium if any showtime exists referring to it. You cannot delete a movie if any showtime exists referring to it
- Base ticket price is a float expected in rm5 - rm500.
- Discount is a percentage expected in 0-50% and stored as a fraction (e.g., .2). There is a universal discount across all movies.
- Data is written to a text file in sections of (movies, showtimes, auditoriums, tickets).
- Showtimes.txt is written and if it exists overwrites showtimes when read in.
- Each manager menu loops until "exit to main menu". Each sub-menu supports add/update/remove/list + back.
- "statistics" merely prints current listings + pricing/discount math without any mention of seats/customers/payments.
- Many inputs attempt limited retries and show friendly errors like, "invalid choice, try again

Flowchart / Pseudocode

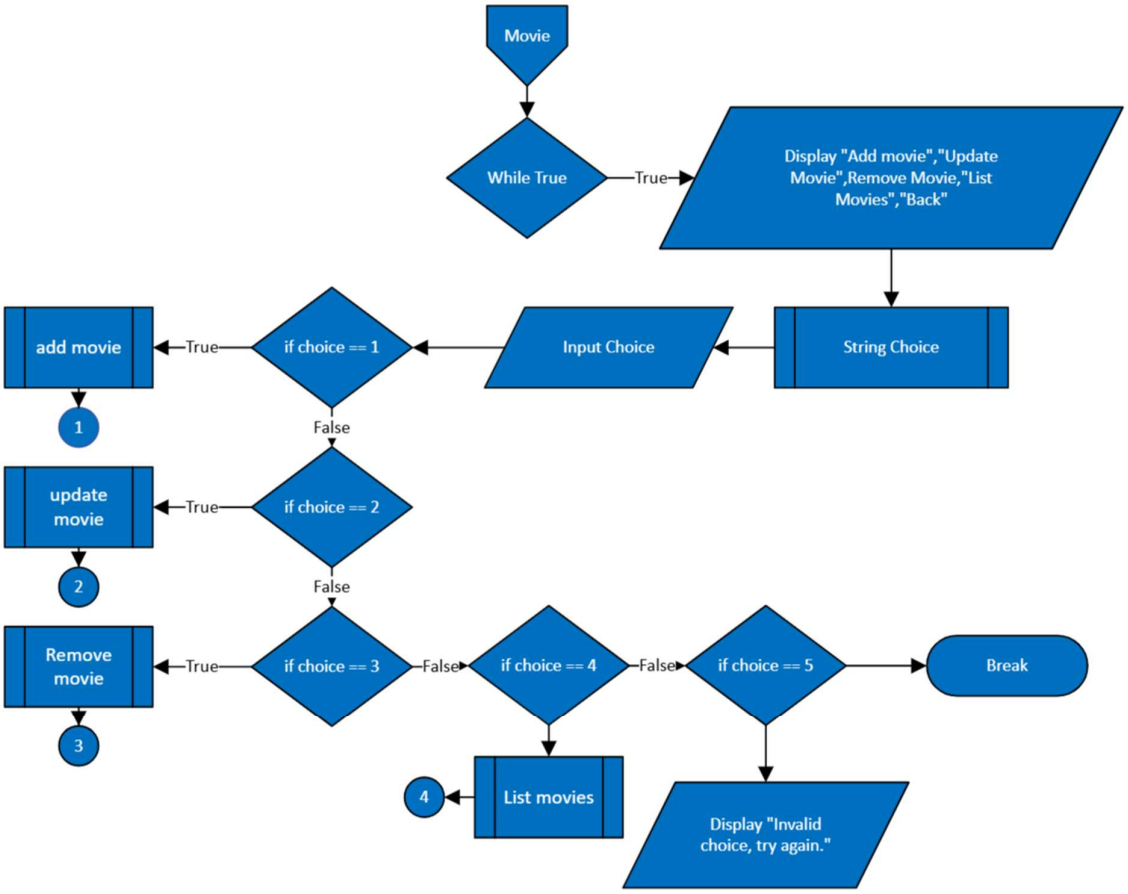
Main Menu

The cinema manager displays a perpetual main menu at the start. The user—aka the manager—can choose between movies, auditoriums, showtimes, prices & discounts, or statistics and can return to the main menu at any time. Invalid selections result in a printout of an error message and prompt the user again.

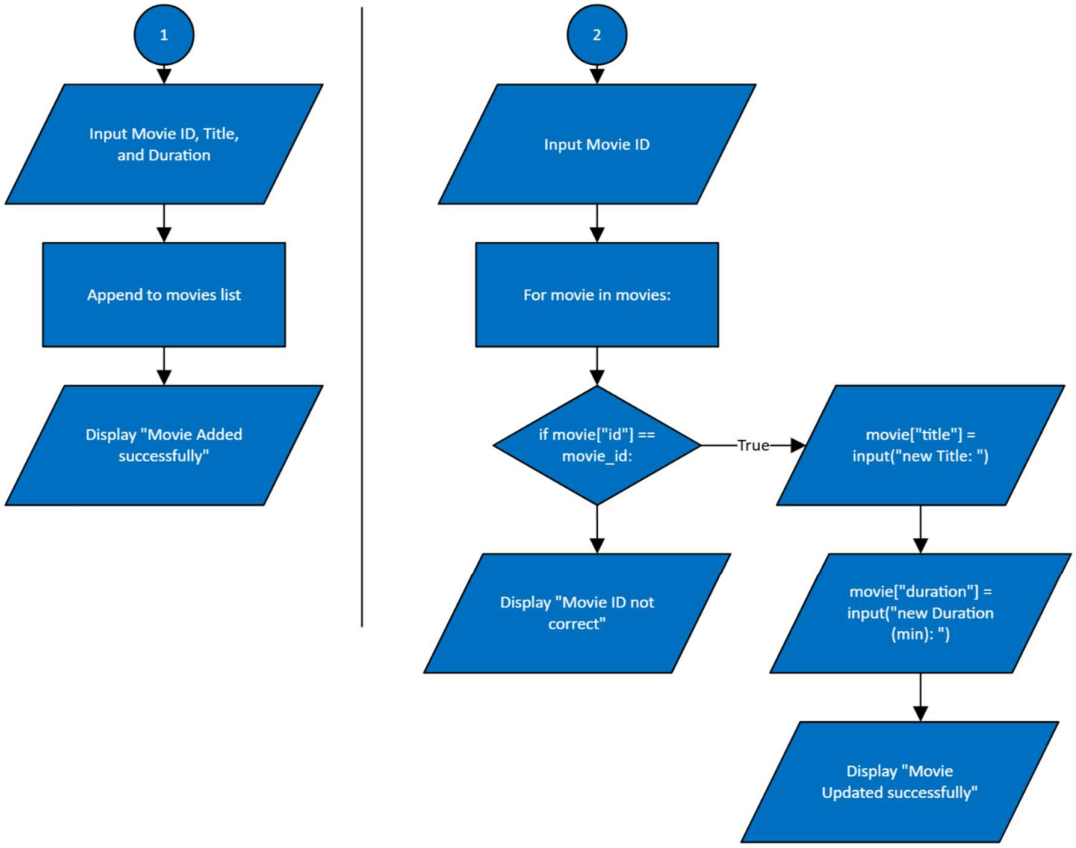


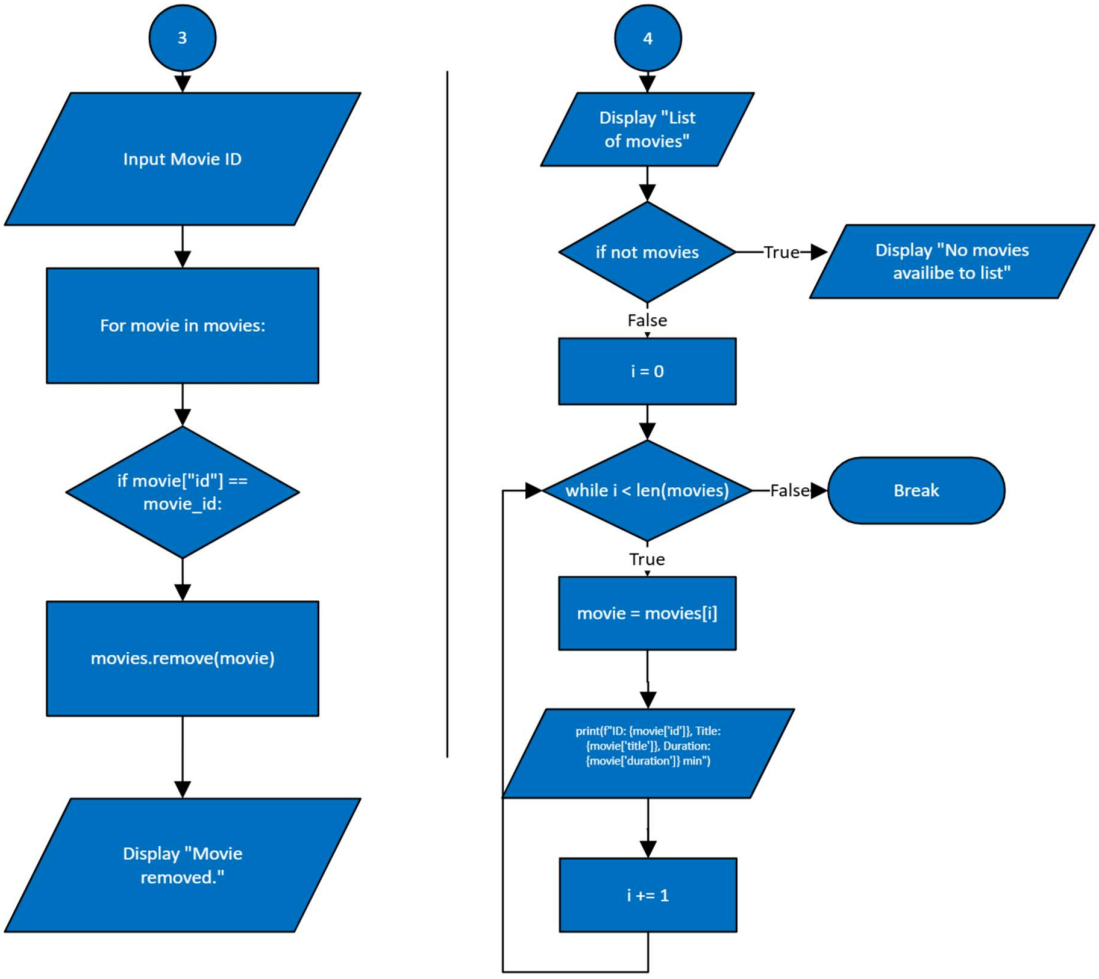
Movie Management Submenu

For Movie Management, The User Can Create A New Movie By Providing A Title And Duration, Update A Specific Movie Via Movie Id, Delete A Movie No Longer Needed Or List All Existing Movies In The System. Ids Are Automatically Generated In Consecutive Order. In Addition, There Is Input Validation To Ensure Titles Are Of An Appropriate Length And Durations Are Within The Appropriate Limits. If A Movie Has Been Scheduled For Any Showtime, It Cannot Be Deleted Until That Showtime Is Cancelled Or Transferred To Another Movie.



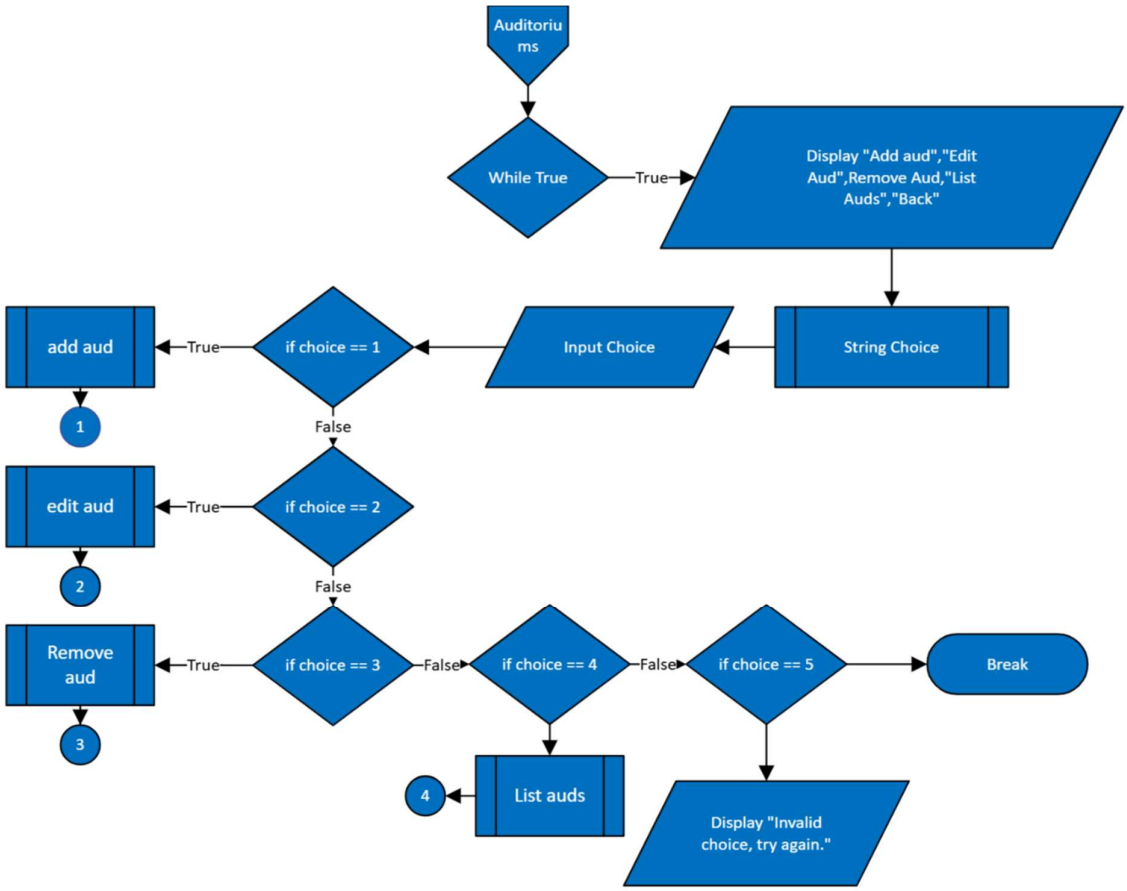
Movie Management Definitions



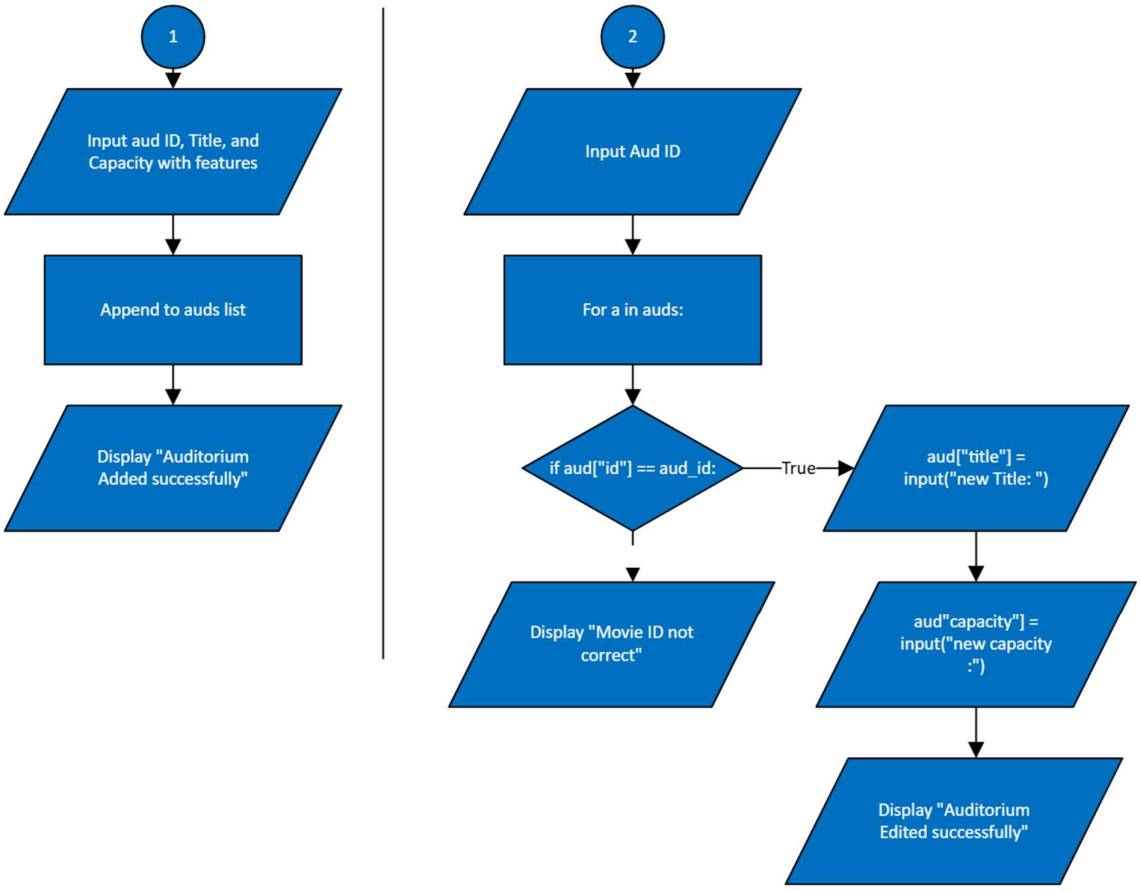

3
Input Movie ID
For movie in movies:
if movie["id"] == movie_id:
movies.remove(movie)
Display "Movie removed."
4
Display "List of movies"
if not movies
True
Display "No movies availibe to list"
False
i = 0
while i < len(movies)
False
Break
True
movie = movies[i]
print(f"ID: {movie['id']}, Title: {movie['title']}, Duration: {movie['duration']} min")
i += 1

Auditorium Management Submenu

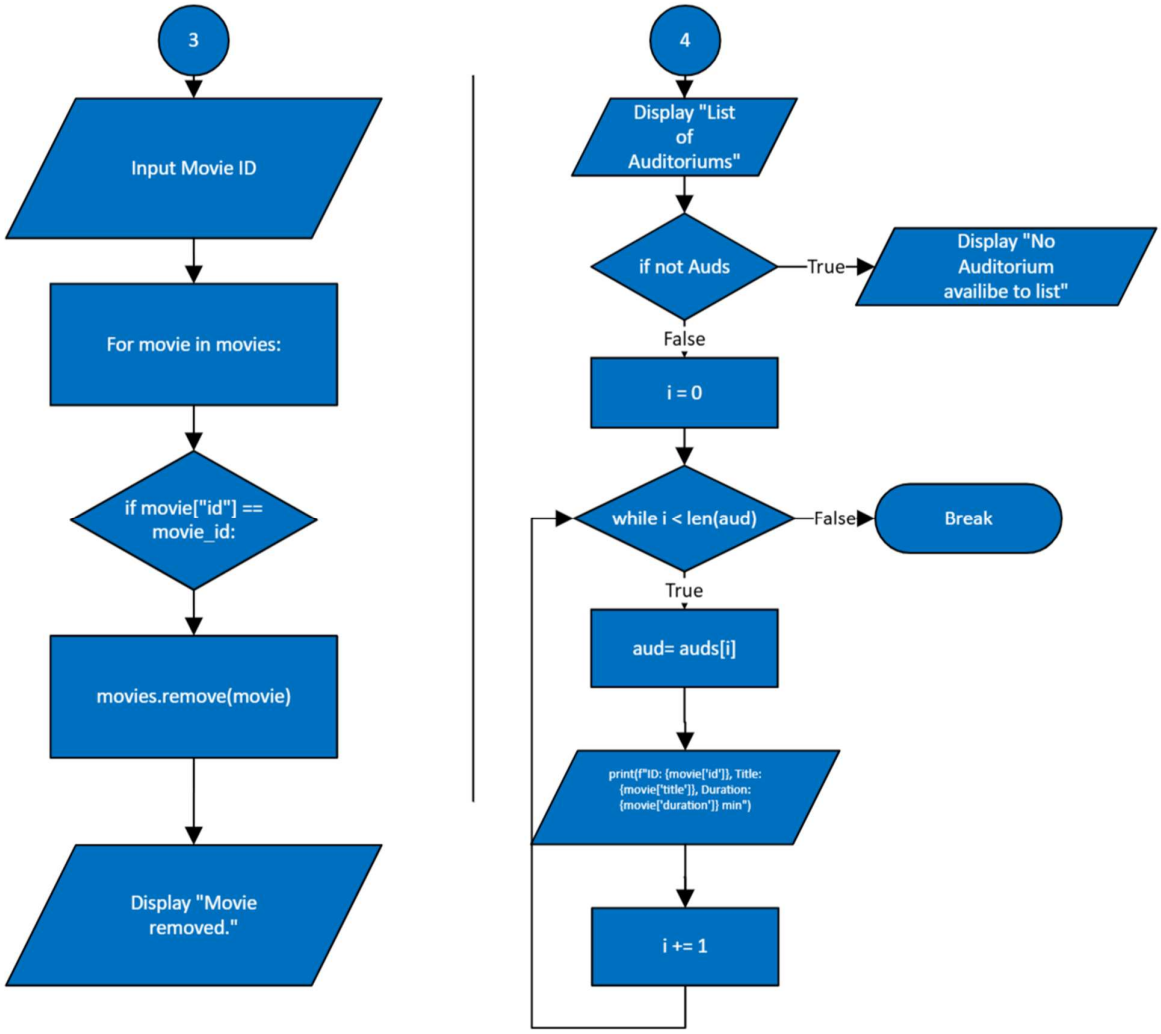
The Auditorium Management Is Done Through A Similarly Structured Aspect. The Manager Creates Auditoriums With An Auto-Generated Id, A Name, And The Size Of Seats Between A Specified Range. These Limited, Required, And Optional Parameters Do Not Necessitate The Addition Of Any Unnecessary Details. The Manager Can Also Update Or Remove Existing Auditoriums, Auditing The List Of All Auditoriums At Any Time. However, Due To Data Integrity And Avoiding Orphan References, An Auditorium Cannot Be Removed While A Showtime Is Using It. The Same Relationship Exists For Auditoriums As It Did For Movies.



Auditorium Menu Definitions

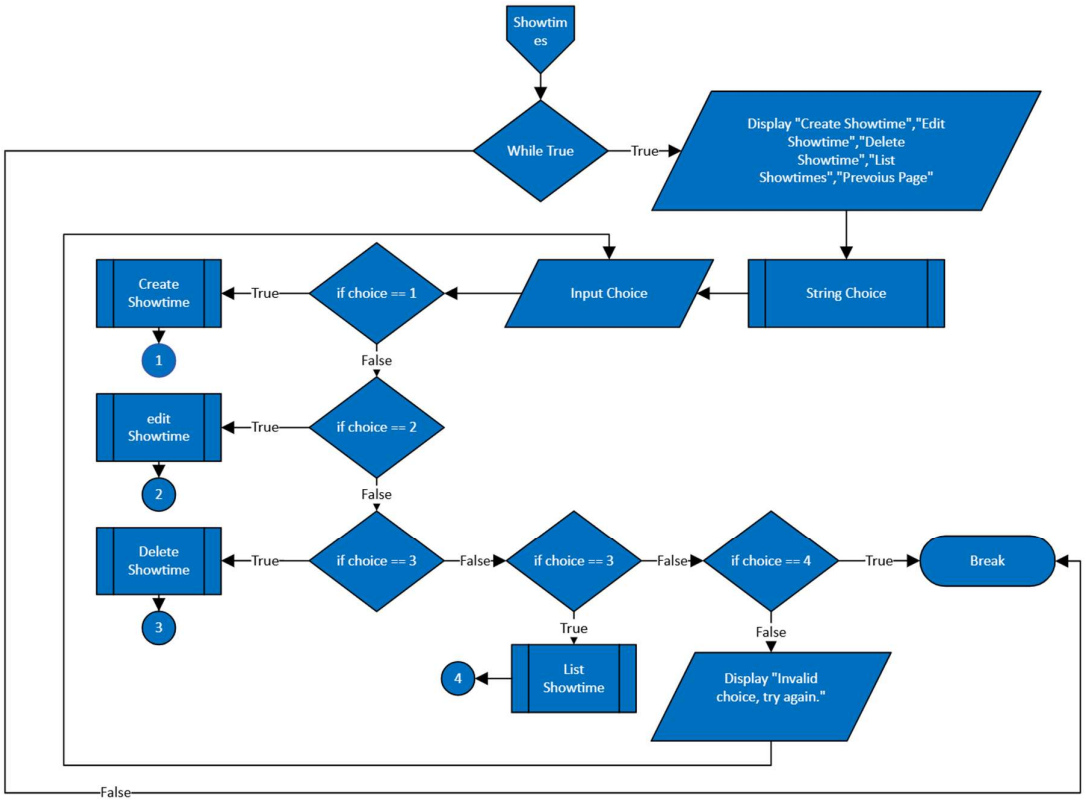


Auditorium Menu Definitions

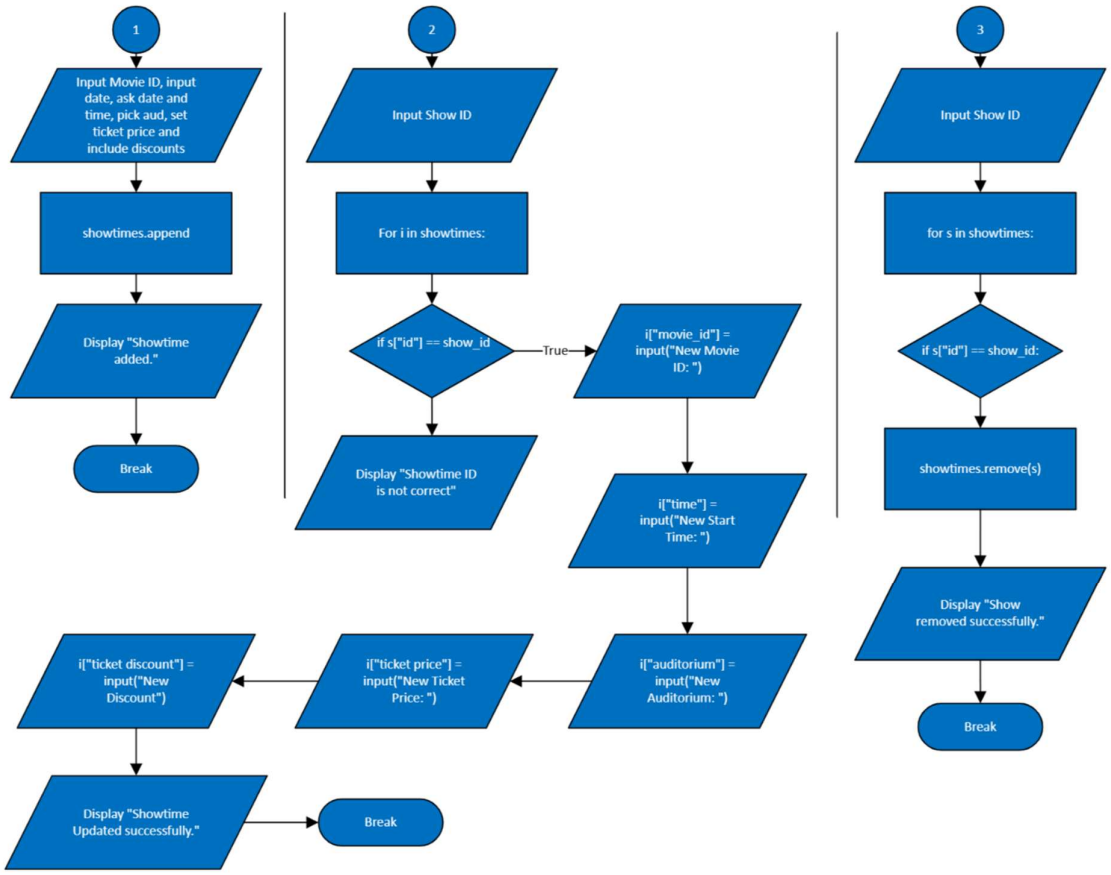


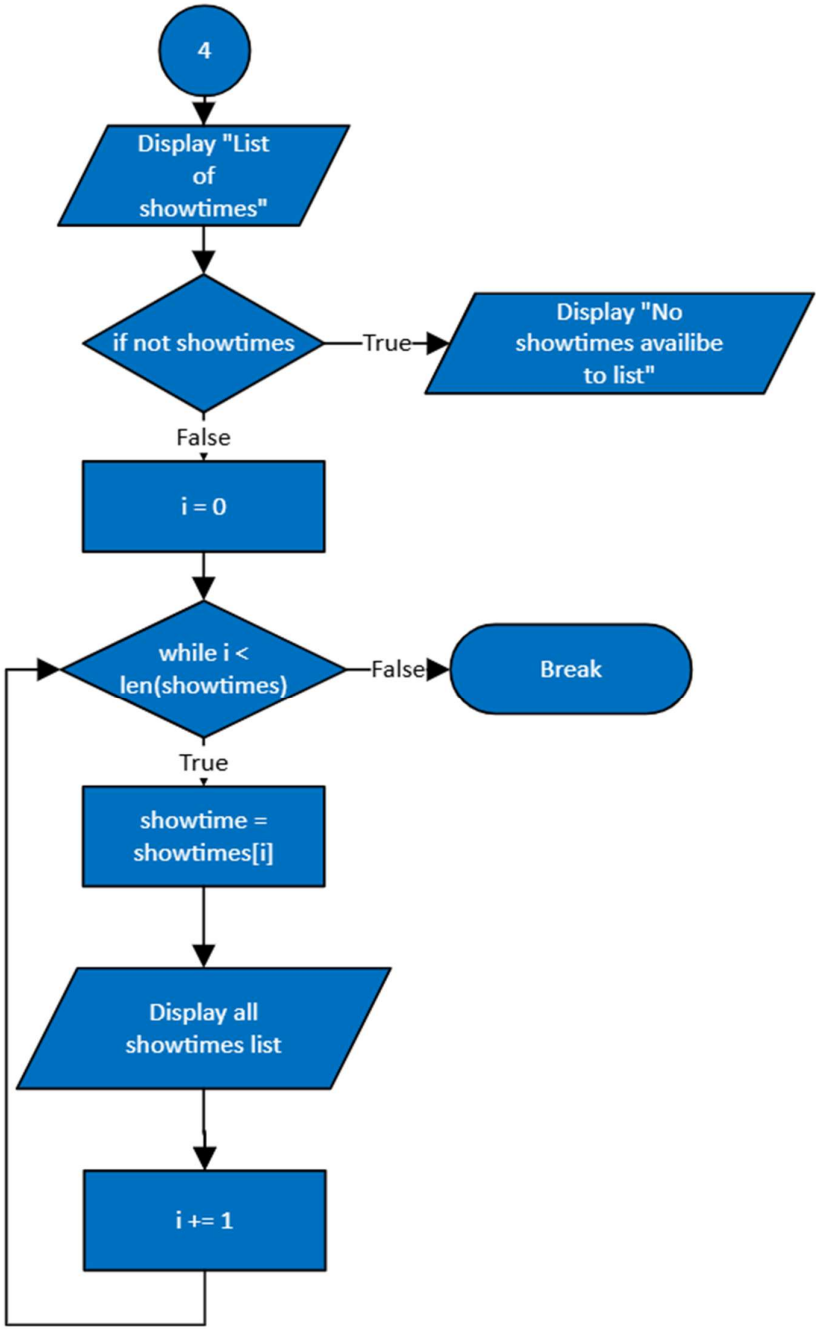
Showtimes Management Submenu

The Showtime Management Component Connects Specific Movies To Auditoriums At Certain Times. Therefore, To Create A Showtime, The Manager Must Select A Valid Movie Id Via The Previously Created List And A Valid Auditorium Id And Enter A Required Start Time (With Am/Pm Formatting) That Falls Within The Stipulated Operating Hours. Showtimes Can Be Edited For Movies/Auditoriums/Times; However, It Is Important To Note That Editing Relies On Current Lists So That Only Valid Ids Can Be Used And The Time Entry Once Again Falls Within Operation Hours. Deleting And Listing Showtimes Is Also A Simple Process Via Id.



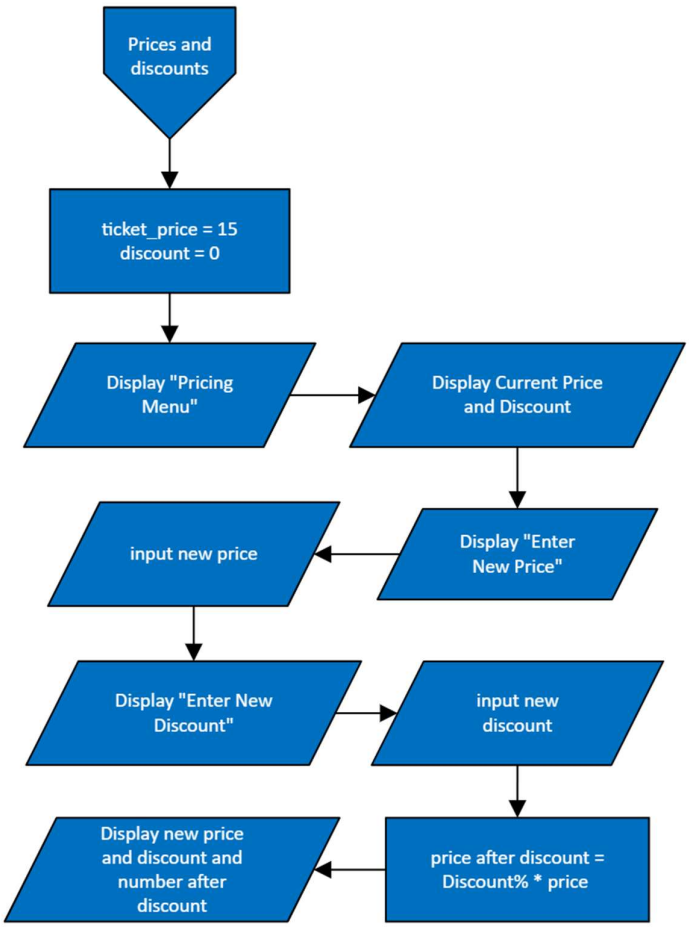
Showtimes Menu Definitions




4
Display "List of showtimes"
if not showtimes
True
Display "No showtimes availibe to list"
False
i = 0
while i < len(showtimes)
False
Break
True
showtime = showtimes[i]
Display all showtimes list
i += 1

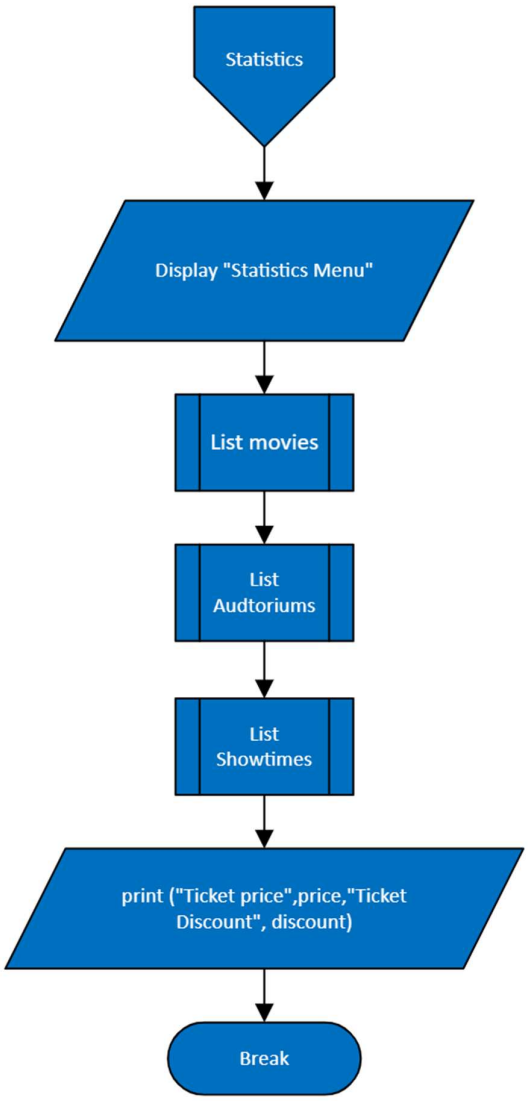
Price And Discounts Management Submenu

When It Comes To Prices & Discounts, Ticket Pricing Is Afforded Throughout The System. A Base Ticket Price Can Be Entered As A Numeric Value That Falls Within Various Defined Ranges. A Global Discount Can Also Be Established As A Percentage But Only One Universal Value May Exist. This Is Important Because, As Mentioned Before, Elsewhere, This Will Play A Role In Considerations For Totals/Costs That Require Accurate Percentages Calculated For Uniformity. Therefore, This Discount Must Be Associated With The System Configuration.



Statistics Submenu

Lastly, Statistics Is A Simple Way For The Manager To View A Summary Of What's Going On In The System At Any Time Which Indicates Its Current State. The Manager Can View The Current Lists Of Movies, Auditoriums, And Showtimes, Along With The Current Ticket Price And Discount, And What That Equates To For An Effective Price. This Helps The Manager Understand What They've Recently Done And That They've Saved Anything To This Point.



Explanation Of Code

Explanation of Code

Application Of Different Types Of Storage

Storage Types Will Be Used Throughout The Project To Manage And Separate Data Effectively. There Are Multiple Types Used For Different Purposes, But The Primary Ones In This Project Are Lists, Dictionaries, And Text Files.

Lists

```
movies = []
showtimes = []
auditoriums = []

loaded_showtimes = []
```

Dictionaries

The Dictionaries Themselves Will Store Multiple Pieces Of Like Information:

```
loaded_showtimes.append({"id": sid,"movie_id": movie_id,"time": time,"auditorium_id": aud_id})

movies.append({"id": movie_id,"title": title,"duration": duration})

auditoriums.append({"id": aid, "name": name, "capacity": capacity})
```

Therefore, By Using A List Of Dictionaries In A Real-World Application Program With A Dynamic User Aspect, It Allows For Easy Access To Creation, Searching, Updating, And In-Place Deletion Since Each Dictionary Is A Record In A List With Recognized Keys.

Text Files

```
Data_File = "Cinema_Data.Txt"
Showtimes_File = "Showtimes.Txt"
```

All Movie Data Will Be Stored Through Persistent Storage Via A Text File. For This Program, It Will Be Cinema_Data.Txt, And A Separate File For The Ticketing Clerk And Others Who Need The Data Of Showtimes Alone. The Original Program Assumes This Data Is Created The First Time It's Run. In The Process Of Saving Data:

```
f = open(data_file, "w")

f.write("--MOVIES--\n")
for m in movies:
    f.write(str(m["id"]) + "|" + m["title"] + "|" + str(m["duration"]) + "|" + "\n")
```

And

```
f = open(data_file, "r")
section = ""
#removing the hidden newlines
for line in f:
    line = line.replace("\n", "")
    if line == "":
        continue
```

...

```
except FileNotFoundError:
    print("The file is not created yet, a new one will be created.")
except:
    print("Error")
```

To Load The Data:

Therefore, This Storage Means Of Non-Volatile Storage Is A Form Of A Flat-File Database That Exists In A Simplified Use Case Because Sections Exist For Human-Readable Separation Such As

`--Movies--, --Showtimes--, --Auditoriums—`

So It's Easier To Manage.

Tuples Weren't A Good Choice Because This Application Requires Mutable, Named Fields Where You Can Change, Remove, And Serialize—Tuples Are **Immutable** And Positional,

Meaning It's Not As Simple As Lists And Much More Error-Prone. In Addition, Had We Gone With Tuples, We'd Have To Recreate Each Record On Every Change, Tracking Fields By Index And At Risk Of Creating Organizational Chaos And Invalid Saves And Loads.

Application Of Control Structures

Control Structures Are Responsible For Managing The Flow Of Information In The Program. There Are Two Types Used In This Project: Selection And Iteration.

• **Selective Structures** (**`If, Elif, Else`**)

Control Structures Allow Conditional Statements To Determine What A User May Need Based On Input.

In This Program, For Example:

```
if entered_choice == "1":
    movies_menu()
elif entered_choice == "2":
    showtimes_menu()
elif entered_choice == "3":
    auditoriums_menu()
elif entered_choice == "4":
    price_menu()
elif entered_choice == "5":
    statistics_menu()
elif entered_choice == "6":
    print("Exiting Cinema Management Menu...")
    save_data()
    break
else:
    print("Invalid choice, try again.")
```

This Is Massively Relied Upon Both In Menus And Validation Checks To Determine What The User Hopes To Do (And If It's Possible) - For Example Validating An Id Or Not Being Able To Remove Something If An Auditorium Is Linked To A Movie.

Nested If Statements Also Exist For Additional Validation Purposes (I.E., Confirming Type Integrity Before Range Checking).

```
if len(title) < 1 or len(title) > 100:
    print("Title can't be less than one letter and more than 100 letters")
    return
#limiting the times the end-user can add a not proper input
tries = 0
while tries < 3:
    try:
        duration = int(input("Enter Duration (minutes): "))
        if duration >= 30 and duration <= 300:
            break
        else:
            #a realistic duration limit for movies
            print("Duration must be between 30 and 300 minutes.")
            tries = tries + 1
```

• **Iterative Structures** (**`For`** Loops And **`While`** Loops)

Loops Allow Information To Be Processed Until Something Happens. For A Menu-Driven Program Like This One It's Critical; **While** Loops Allow The Menu Selections To Continue Until Something Is Selected While Providing Repeated Input Opportunities Via:

```
tries = 0
while tries < 3:
    try:
        duration = int(input("Enter Duration (minutes): "))
        if duration >= 30 and duration <= 300:
            break
        else:
            #a realistic duration limit for movies
            print("Duration must be between 30 and 300 minutes.")
            tries = tries + 1
    except:
        print("Please enter a whole number for minutes.")
        tries = tries + 1
```

```
for movie in movies:
    if movie["id"] == movie_id:
        movies.remove(movie)
        print("Movie removed.")
        save_data()
```

And

```
for a in auditoriums:
    print(f"ID: {a['id']}, Name: {a['name']}, Capacity: {a['capacity']}")
```

Are Example Of A Loop Through The **Stored Data** To Easily Iterate What Was Stored To Print Or Process Every Record.

Application Of Try–Except

The **Try-Except** Structure Allows Users To Attempt Operations Without Crashing Due To Runtime Errors. Therefore, Protected Operations Are Anticipated With Logic Under Stable Conditions But Accounting For Predictable Errors Such As No File Found Or User Error (**Invalidation**).

```
try:
    capacity = int(input("Enter Capacity (number): "))
    if capacity < 30 or capacity > 1000:
        print("The Capacity Of Auditoriums Can't be more than 1000 or less than 30
        return
except:
    print("Invalid capacity.")
    return
```

This Code Checks That The Amount Entered For The Auditorium Capacity Is Valid And Does Not Exceed Maximum Limits. It Is Important To Note That If The Value Is Invalid Or Below **30**/Above **1000**, The Program Does Not Crash, It Simply Reports An Error.

Without A **`Try-Except`** Here, If The User Typed A Letter Instead Of A Number It Would Crash The Program. Instead It Safely Asks Them To Try Again. This Is An Example Of Defensive Programming As It Projects Potential Errors Caused By Users Before They Crash The Program From Failure.

```
try:
    user_time = datetime.strptime(time_input, "%I:%M%p")
except:
    print("Invalid time format.")
    tries += 1
    continue
```

Application Of Validation

Validation Ensures That Inputs Made By Users Are Expected And Follow Anticipated Rules And Logical Boundaries Throughout The Project.

• Validation Example #1 **(Movies)**

```
def update_movie():
    movie_id = input("Enter Movie ID to update: ")
    for movie in movies:
        if movie["id"] == movie_id:
            new_title = input("New Title: ").strip().upper()
            if len(new_title) < 1 or len(new_title) > 100:
                print("Title can't be less than one letter and more than 100 letters")
                return
            movie["title"] = new_title
```

• Validation Example #2 (**Showtimes**)

```
show_id = input("Enter Showtime ID to edit: ").strip().upper()
for s in showtimes:
    if s["id"] == show_id:
```

• Validation Example #3 (**Auditoriums**)

```
found = False
for a in auditoriums:
    if a["id"] == aud_id:
        found = True
        break
```

Validation Ensures That Data In The Program Is Always Realistic - Preventing Impossible Showings For Films, Negative Durations/Existing Times Or Zero Capacities. Validation Is One Of The Most Common Habits Used In Real World Applications/Programs Since Without It There's No Accountability For Poor User Input.

Flow Of Add Functions

The System Has Multiple Add Functions For Movies, Auditoriums And Showtimes All Dedicated To Each Major Category Represented Within The System.

An Example Would Be Id Creation Generation - Every New Record Requires A Unique Id That's Automatically Assigned So That Manual Entry Isn't Necessary And Duplication Isn't Possible.

```
def add_movie():
    global movies

    # Auto generating IDs, max number quantity is 99
    if len(movies) == 0:
        movie_id = "M001"
    else:
        last = movies[-1]["id"]
        try:
            num = int(last[1:]) + 1
        except:
            num = len(movies) + 1

        if num < 10:
            movie_id = "M00" + str(num)
        elif num < 100:
            movie_id = "M0" + str(num)
        else:
            movie_id = "M" + str(num)

    title = input("Enter Title: ").strip().upper()
```

```python
def add_auditorium():
    global auditoriums

    # Auto-generate ID, starting from A001
    if len(auditoriums) == 0:
        aid = "A001"
    else:
        last = auditoriums[-1]["id"]
        try:
            num = int(last[1:]) + 1
        except:
            num = len(auditoriums) + 1

        if num < 10:
            aid = "A00" + str(num)
        elif num < 100:
            aid = "A0" + str(num)
        else:
            aid = "A" + str(num)

    name = input("Enter Auditorium Name: ").strip().upper()
    if len(name) < 1 or len(name) > 100:
        print("Auditorium Name can't be less than one letter and more than 100 letters")
        return
```

This Process Demonstrates Encapsulation And Automation Since Once Someone Runs The Function Everything Necessary To Take Place From Collecting Input, Validating Data, Assigning An Id And Saving To File - Happens Without User Involvement.

Flow Of Update / Edit Functions

Update Functions Require Existing Fields To Be Found Through An Id Before They Can Be Updated/Changed Through User Input.

Example:

```
def update_movie():
    movie_id = input("Enter Movie ID to update: ")
    for movie in movies:
        if movie["id"] == movie_id:
```

If The User Id Didn't Exist However:

```
print("Movie not found.")
```

This Demonstrates Searching And Replacing Within A Field/Movie That Exists Which Is Vital To Success As It's Practical Use Can Be Applied In Real-World Situations When Dynamic Data Is Necessary To Make Changes.

This Also Reinforces Reusability Since The Same Logic Structure Works For All Types Across Three Various Categories.

Flow Of Delete Functions

Delete Functions Are As Simple As Possible But With Safeguards.

```
for movie in movies:
    if movie["id"] == movie_id:
        movies.remove(movie)
        print("Movie removed.")
        save_data()
        return
```

Safeguards Up Front Validate That Nothing Can Be Deleted If It's Linked To Something Else; Potential Errors Linked With Logic Check Against Valid Conditions Prevent Logical Fallacies From Occurring With Poor Data Integrity.

```
if any(s["movie_id"] == movie_id for s in showtimes):
    print("Cannot remove movie; it is used by existing showtimes.
    return
```

Flow Of Generating Reports (Statistics Menu)

The Reports Feature Of The System Was Condensed Into A Statistics Section Where Information From All List Definitions (Movies/Auditoriums/Showtimes) Was Printed And Ticketing Information Was Displayed

```
def statistics_menu():
    print("\n--- Statistics Section ---")

    print("\n--- Movies ---")
    list_movies()

    print("\n--- Auditoriums ---")
    list_auditoriums()

    print("\n--- Showtimes ---")
    list_showtimes()

    print("\n--- Tickets ---")
    print(f"Ticket Price: RM{ticket_price}")
    print(f"Discount: {discount * 100}%")
    print(f"Price After Discount: RM{ticket_price * (1 - discount)}")

    print("\nReturning To Cinema Management Menu...")
    save_data()
```

This Demonstrates Function Reusability (Calling Already Existing List Functions) To Print Within Properly Formatted Means Where Data Is Aggregated Via Common Module Sections Most Like A Real-World Manager Dashboard That Summarizes System Situation In One Output.

Additional Feature(S)

There Are Multiple Additional Features Utilized Throughout This Project To Create Comprehensive Improvements That Would Otherwise Go Unnoticed Without Referencing The Source Code:

- Automatic Id Generation- Ids Such As **M001, A001, S001** Are Automatically Generated Using List Lengths Built On Existing Conditions Which Reduces Possible Errors And Helps Maintain Proper Order Instead Of Creating Any Gaps.
- **Dynamic Prices** - Based On Price Shifts In Ticketing Options There Are Discounts Calculated As Final Prices Dynamically Adjust:

Final_Price = Ticket_Price * (1 - Discount)

- **Persistent Save/Load System**
  Any Time There Are Additions/Changes Made Or Deletions Present/New Modifications **Save_Data()** Must Be Called Immediately To Ensure Proper Performance Doesn't Result In Complete Loss.
- **Validation/User-Friendliness**
  User Can Make An Error But Should Be Allowed A Second Chance Via While Loops For Any Invalid Input.

Screenshot Of Input & Output Of The Code

(Main Menu) First Output

```
--- Cinema Manager Menu ---
1. Manage Movies
2. Manage Showtimes
3. Manage Auditoriums
4. Manage Prices & Discounts
5. Statistics
6. Exit To Main Menu
Enter your choice:
```

Output For **Movie Menu** Option

```
--- Movies Menu ---
1. Add Movie
2. Update Movie
3. Remove Movie
4. List Movies
5. Back
Enter your choice:
```

Output For **Showtimes Menu** Option

```
--- Showtimes Menu ---
1. Create Showtime
2. Edit Showtime
3. Delete Showtime
4. Back
Enter choice:
```

Output For **Auditorium Menu** Option

```
--- Auditoriums Menu ---
1. Add Auditorium
2. Update Auditorium
3. Remove Auditorium
4. List Auditoriums
5. Back
Enter your choice:
```

Output For **Price Menu** Option

```
--- Price Menu ---
Current ticket price: RM 15.0
Current discount: 0.0 %
Enter new base ticket price:
```

Output For **Statistics Menu** Option

```
--- Statistics Section ---

--- Movies ---

List of Movies:
ID: S001, Title: Name, Duration: 80 min
ID: S002, Title: You, Duration: 180 min
ID: S003, Title: Sea Pirates, Duration: 230 min

--- Auditoriums ---

Auditoriums:
ID: A001, Name: Max-1, Capacity: 101
ID: A002, Name: Max-2, Capacity: 202
ID: A003, Name: Max-3, Capacity: 303

--- Showtimes ---

Showtimes List:
ID: SH002, Movie ID: S002, Time: 10:00 PM, Auditorium ID: A002
ID: SH003, Movie ID: S001, Time: 9:30 AM, Auditorium ID: A002
ID: SH004, Movie ID: S001, Time: 12:00 AM, Auditorium ID: A003
ID: SH005, Movie ID: S001, Time: 12:00 PM, Auditorium ID: A002

--- Tickets ---
Ticket Price: RM15.0
Discount: 0.0%
Price After Discount: RM15.0

Returning to Manager Menu...
```

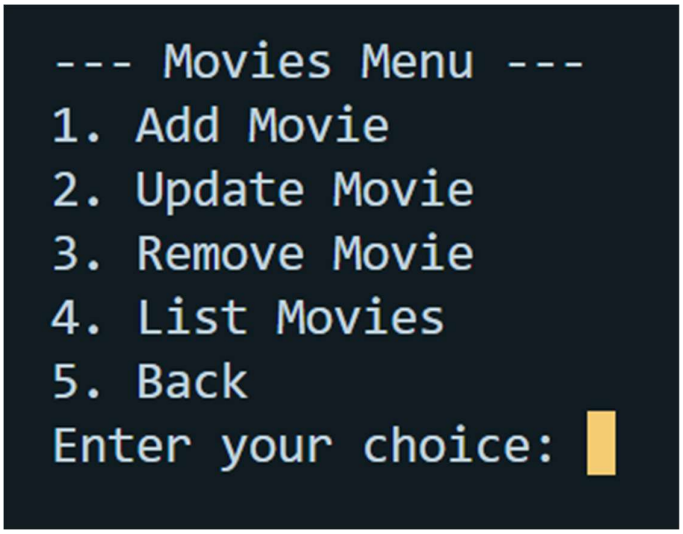
Output For **Exiting Menu** Option

```
Exiting Cinema Manager Menu...
PS C:\Users\azezm\folder\python-Assignment-part1-ver1>
```

Output For A **Wrong** Input

```
--- Cinema Manager Menu ---
1. Manage Movies
2. Manage Showtimes
3. Manage Auditoriums
4. Manage Prices & Discounts
5. Statistics
6. Exit To Main Menu
Enter your choice: 8
Invalid choice, try again.
```

(Movies Menu)



Output For **Add Movie** Option

```
--- Movies Menu ---
1. Add Movie
2. Update Movie
3. Remove Movie
4. List Movies
5. Back
Enter your choice: 1
Enter Title: Demon Slayer
Enter Duration (minutes): 220
Movie added successfully. Movie ID: S004
```

Output For **Update Movie** Option

```
--- Movies Menu ---
1. Add Movie
2. Update Movie
3. Remove Movie
4. List Movies
5. Back
Enter your choice: 2
Enter Movie ID to update: S002
New Title: Chainsaw Man 2
New Duration (minutes): 230
Movie updated.
```

Output For **Remove Movie** Option

```
--- Movies Menu ---
1. Add Movie
2. Update Movie
3. Remove Movie
4. List Movies
5. Back
Enter your choice: 3
Enter Movie ID to remove: S002
Movie removed.
```

Output For **List Movies** Option

```
--- Movies Menu ---
1. Add Movie
2. Update Movie
3. Remove Movie
4. List Movies
5. Back
Enter your choice: 4

List of Movies:
ID: S001, Title: Name, Duration: 80 min
ID: S003, Title: Sea Pirates, Duration: 230 min
ID: S004, Title: Demon Slayer, Duration: 220 min
```

Output For **Back** Option

```
--- Movies Menu ---
1. Add Movie
2. Update Movie
3. Remove Movie
4. List Movies
5. Back
Enter your choice: 5

--- Cinema Manager Menu ---
1. Manage Movies
2. Manage Showtimes
3. Manage Auditoriums
4. Manage Prices & Discounts
5. Statistics
6. Exit To Main Menu
Enter your choice:
```

Output For A **Wrong** Input

```
--- Movies Menu ---
1. Add Movie
2. Update Movie
3. Remove Movie
4. List Movies
5. Back
Enter your choice: g
Invalid choice, try again.
```

(Showtimes Menu)

```
--- Showtimes Menu ---
1. Create Showtime
2. Edit Showtime
3. Delete Showtime
4. Back
Enter choice:
```

Output For **Create Showtime** Option

```
--- Showtimes Menu ---
1. Create Showtime
2. Edit Showtime
3. Delete Showtime
4. Back
Enter choice: 1
Enter Movie ID: S001
---Examples: 9am, 9:30pm, 11:45pm
---Note: Operation Hours Are Between 8am to 12pm
Enter time: 5pm

Available Auditoriums:
- A001 Max-1
- A002 Max-2
- A003 Max-3
Enter Auditorium ID: A001
Showtime added. ID: SH006 Movie: S001 Auditorium: A001 Time: 5:00 PM
```

Output For **Edit Showtime** Option

```
--- Showtimes Menu ---
1. Create Showtime
2. Edit Showtime
3. Delete Showtime
4. Back
Enter choice: 2
Enter Showtime ID to edit: SH002
New Movie ID: s002
Enter New Start Time:
---Examples: 9am, 9:30pm, 11:45pm
---Note: Operation Hours Are Between 8am to 12pm
Enter time: 5:30pm

Available Auditoriums:
- A001 Max-1
- A002 Max-2
- A003 Max-3
New Auditorium ID (press Enter to keep current):
Showtime updated.
```

Output For **Delete Showtime** Option

```
--- Showtimes Menu ---
1. Create Showtime
2. Edit Showtime
3. Delete Showtime
4. Back
Enter choice: 3
Enter Showtime ID to delete: SH003
Showtime deleted.
```

Output For **Back Showtime** Option

```
--- Showtimes Menu ---
1. Create Showtime
2. Edit Showtime
3. Delete Showtime
4. Back
Enter choice: 4

--- Cinema Manager Menu ---
1. Manage Movies
2. Manage Showtimes
3. Manage Auditoriums
4. Manage Prices & Discounts
5. Statistics
6. Exit To Main Menu
Enter your choice:
```

(Auditoriums Menu)

```
--- Auditoriums Menu ---
1. Add Auditorium
2. Update Auditorium
3. Remove Auditorium
4. List Auditoriums
5. Back
Enter your choice:
```

Output For **Add Auditorium** Option

```
--- Auditoriums Menu ---
1. Add Auditorium
2. Update Auditorium
3. Remove Auditorium
4. List Auditoriums
5. Back
Enter your choice: 1
Enter Auditorium Name: Max-3
Enter Capacity (number): 303
Auditorium added successfully. Auditorium ID: A005
```

Output For **Update Auditorium** Option

```
--- Auditoriums Menu ---
1. Add Auditorium
2. Update Auditorium
3. Remove Auditorium
4. List Auditoriums
5. Back
Enter your choice: 2
Enter Auditorium ID to update: A006
New Auditorium Name: Max-super
New Capacity (number): 450
Auditorium updated.
```

Output For **Remove Auditorium** Option

```
--- Auditoriums Menu ---
1. Add Auditorium
2. Update Auditorium
3. Remove Auditorium
4. List Auditoriums
5. Back
Enter your choice: 3
Enter Auditorium ID to remove: A005
Auditorium removed.
```

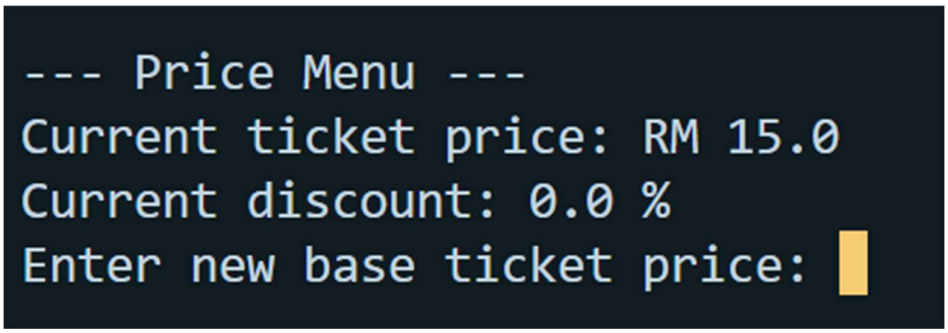
Output For **List Auditorium** Option

```
--- Auditoriums Menu ---
1. Add Auditorium
2. Update Auditorium
3. Remove Auditorium
4. List Auditoriums
5. Back
Enter your choice: 4

Auditoriums:
ID: A001, Name: Max-1, Capacity: 101
ID: A002, Name: Max-2, Capacity: 202
ID: A003, Name: Max-3, Capacity: 303
ID: A004, Name: 1, Capacity: 100
ID: A006, Name: Max-super, Capacity: 450
```

Output For **Back** Option

```
--- Auditoriums Menu ---
1. Add Auditorium
2. Update Auditorium
3. Remove Auditorium
4. List Auditoriums
5. Back
Enter your choice: 5

--- Cinema Manager Menu ---
1. Manage Movies
2. Manage Showtimes
3. Manage Auditoriums
4. Manage Prices & Discounts
5. Statistics
6. Exit To Main Menu
Enter your choice:
```

(Price Menu)



Output For **New Price** & **New Discount**

```
--- Price Menu ---
Current ticket price: RM 15.0
Current discount: 0.0 %
Enter new base ticket price: 15
New Ticket price set to: RM 15.0
Enter discount (between 0 and 1, for example 0.2 = 20%): 20
Discount must be between 0 and 1.0
Enter discount (between 0 and 1, for example 0.2 = 20%): 0.2
Discount set to: 20.0 %
Prices updated. Base: RM 15.0 , Discount: 20.0 %
Final price after discount: RM 12.0
```

(Statistics Menu)

```
--- Statistics Section ---

--- Movies ---

List of Movies:
ID: S001, Title: Name, Duration: 80 min
ID: S003, Title: Sea Pirates, Duration: 230 min
ID: S004, Title: Demon Slayer, Duration: 220 min

--- Auditoriums ---

Auditoriums:
ID: A001, Name: Max-1, Capacity: 101
ID: A002, Name: Max-2, Capacity: 202
ID: A003, Name: Max-3, Capacity: 303
ID: A004, Name: 1, Capacity: 100
ID: A006, Name: Max-super, Capacity: 450

--- Showtimes ---

Showtimes List:
ID: SH002, Movie ID: s002, Time: 5:30 PM, Auditorium ID: A002
ID: SH004, Movie ID: S001, Time: 12:00 AM, Auditorium ID: A003
ID: SH005, Movie ID: S001, Time: 12:00 PM, Auditorium ID: A002
ID: SH006, Movie ID: S001, Time: 5:00 PM, Auditorium ID: A001

--- Tickets ---
Ticket Price: RM15.0
Discount: 20.0%
Price After Discount: RM12.0

Returning to Manager Menu...
```

(Exiting To Main Cinema Menu)

```
--- Cinema Manager Menu ---
1. Manage Movies
2. Manage Showtimes
3. Manage Auditoriums
4. Manage Prices & Discounts
5. Statistics
6. Exit To Main Menu
Enter your choice: 6
Exiting Cinema Manager Menu...
PS C:                    ython-Assignment-part1-ver1>
```

Text Files Screenshots

- Cinema_Data.Txt

```
--MOVIES--
M001|Transformers The Last Knight|80|
M002|Terminator 5|230|
M003|Demon Slayer|220|
M004|Fast and Furious X|90|
--SHOWTIMES--
S001|M002|5:30 PM|A002
S002|M001|12:00 AM|A003
S003|M001|12:00 PM|A002
S004|M001|5:00 PM|A001
S005|M003|10:00 AM|A003
--AUDITORIUMS--
A001|Max-1|101
A002|Max-2|202
A003|Max-3|303
--TICKETS--
25.0|0.5
```

- Showtimes.Txt

```
--SHOWTIMES--
S001|M002|5:30 PM|A002
S002|M001|12:00 AM|A003
S003|M001|12:00 PM|A002
S004|M001|5:00 PM|A001
S005|M003|10:00 AM|A003
```

References

Data, R. (2025). *W3schools*. Retrieved From w3schools.Com: Https://Www.w3schools.Com/Python/Python_Try_Except.Asp

Data, R. (2025). *W3schools*. Retrieved From w3schools.Com: Https://Www.w3schools.Com/Python/Python_File_Handling.Asp

Data, R. (2025). *W3schools*. Retrieved From w3schools.Com: Https://Www.w3schools.Com/Python/Python_Datetime.Asp

Geeksforgeeks. (2025, October 04). Retrieved From Geeksforgeeks.Org: Https://Www.Geeksforgeeks.Org/Python/Python-Functions/